Proxy Service

A comprehensive, containerized proxy service with VPN routing, intelligent caching, and network-wide access for all devices.

Features

- **HTTP/HTTPS Proxy**: Squid-based caching proxy for web traffic
- **SOCKS5 Proxy**: Dante SOCKS proxy for flexible protocol support
- **VPN Routing**: Route all proxy traffic through OpenVPN for privacy
- **Intelligent Caching**: Reduce bandwidth by caching frequently accessed content
- **Streaming Cache**: Special handling for video/audio streaming services
- **Network-Wide Access**: Share proxy connection with all devices on your network
- **Auto-Recovery**: Automatic VPN reconnection and service health monitoring
- **Cache Invalidation**: Automatic and manual cache cleanup mechanisms

Quick Start

```bash
# 1. Clone the repository
git clone git@github.com:vasic-digital/Proxy.git
cd Proxy

# 2. Copy and configure environment
cp .env.example .env
# Edit .env with your settings

# 3. Initialize the service
./init

# 4. Start the proxy
./start

# 5. Check status
./status
```

Architecture

```
┌─────────────────────────────────────────────────────────────────
│                         HOST MACHINE
│
├─────────────────────────────────────────────────────────────────
│
│  ┌─────────────────┐     ┌─────────────────┐
│  │   NETWORK       │     │   VPN CLIENT    │───────► VPN Server
│  │   CLIENTS       │     │   (Optional)    │
│  └───────┬─────────┘     └───────┬─────────┘
│          │                       │
│          ▼                       │
│  ┌─────────────────┐             │
│  │  HTTP PROXY     │◄────────────┤
│  │  (Squid:53128)   │
│  │                            │
│  │  + Cache        │             │
│  └───────┬─────────┘             │
│          │                       │
│  ┌───────▼─────────┐             │
│  │  SOCKS PROXY    │◄────────────┘
│  │  (Dante:51080)
│                                              │
│  └─────────────────┘
│
│  ┌─────────────────┐     ┌─────────────────┐
│  │ ADMIN PANEL     │     │ CACHE MGMT      │
│  │ (Caddy:58080)    │     │ (Automated)      │
│  └─────────────────┘     └─────────────────┘
│
```

```
│
└──────────────────────────────────────────────────────────────────
```

Directory Structure

```
Proxy/
├── .env                    # Environment configuration (git-ignored)
├── .env.example            # Configuration template
├── .gitignore              # Git ignore rules
├── AGENTS.md               # AI agent guidelines
├── README.md               # This file
├── USER_GUIDE.md           # Detailed user manual
├── docker-compose.yml      # Container service definitions
├── init                    # Environment initialization script
├── start                   # Start services
├── stop                    # Stop services
├── restart                 # Restart services
├── status                  # Service status checker
├── cache                   # Cache management script
├── lib/
│   └── container-runtime.sh    # Shared runtime functions
├── config/
│   ├── squid/
│   │   └── squid.conf          # Squid proxy configuration
│   ├── dante/
│   │   └── sockd.conf          # SOCKS proxy configuration
│   ├── caddy/
│   │   └── Caddyfile           # Admin interface config
│   └── streaming.conf          # Streaming cache settings
├── scripts/
│   ├── cache-invalidator.sh    # Cache cleanup automation
│   └── vpn-monitor.sh          # VPN health monitoring
├── services/
│   └── admin/
│       └── index.html          # Admin panel interface
├── docs/
│   ├── ARCHITECTURE.md         # System architecture
│   ├── CACHE.md                # Caching documentation
│   ├── VPN.md                  # VPN configuration guide
│   ├── NETWORK_MODES.md        # Network mode comparison
│   ├── TROUBLESHOOTING.md      # Common issues and solutions
│   ├── CLIENT_SETUP.md         # Client setup for all devices
│   ├── ANDROID_TV.md           # Android TV setup guide
│   ├── BROWSERS.md             # Browser configuration
│   └── MOBILE_DEVICES.md       # iOS/Android setup
├── tests/
│   └── run-tests.sh            # Test runner
├── logs/                       # Log files (git-ignored)
```

```
└── Upstreams/
    └── GitHub.sh               # Git upstream configuration
```

Configuration

Environment Variables

Variable	Description	Default
`CONTAINER_RUNTIME`	Runtime to use (podman/docker/auto)	auto
`HTTP_PROXY_PORT`	HTTP/HTTPS proxy port	53128
`SOCKS_PROXY_PORT`	SOCKS5 proxy port	51080
`PROXY_ADMIN_PORT`	Admin panel port	58080
`USE_VPN`	Enable VPN routing	false
`VPN_USERNAME`	VPN provider username	-
`VPN_PASSWORD`	VPN provider password	-
`VPN_OVPN_PATH`	Path to .ovpn config file	-
`CACHE_DIR`	Cache storage directory	./cache
`CACHE_MAX_SIZE_GB`	Maximum cache size	50
`CACHE_MAX_AGE_DAYS`	Max age before invalidation	30

See `.env.example` for complete configuration options.

Usage

Starting the Service

```
# Start with VPN (if configured)
./start

# Start without VPN
./start --no-vpn

# Start with verbose output
./start -v

# Pull latest images before starting
./start --pull
```

Stopping the Service

```
# Stop services
./stop
```

```bash
# Stop and remove containers
./stop --remove

# Stop, remove containers and images
./stop --purge

# Stop and clear cache
./stop --clean-cache
```

Checking Status

```bash
# Basic status
./status

# Detailed status
./status -v

# JSON output
./status --json

# Watch mode (continuous monitoring)
./status --watch
```

Managing Cache

```bash
# Show cache statistics
./cachectl stats

# Clear all cache
./cachectl clear

# Force clear without confirmation
./cachectl clear -f

# Run invalidation (remove stale files)
./cachectl invalidate

# Trim cache to specific size
./cachectl trim 30  # Trim to 30GB
```

Client Configuration

Linux/macOS

```bash
# Set environment variables
export HTTP_PROXY="http://HOST_IP:53128"
export HTTPS_PROXY="http://HOST_IP:53128"
export ALL_PROXY="socks5://HOST_IP:51080"
```

```bash
export NO_PROXY="localhost,127.0.0.1"

# Or add to ~/.bashrc or ~/.zshrc for persistence
```

System-wide (Linux)

```bash
# Add to /etc/environment
HTTP_PROXY="http://HOST_IP:53128"
HTTPS_PROXY="http://HOST_IP:53128"
NO_PROXY="localhost,127.0.0.1"
```

Windows

```powershell
# PowerShell
$env:HTTP_PROXY = "http://HOST_IP:53128"
$env:HTTPS_PROXY = "http://HOST_IP:53128"

# Command Prompt
set HTTP_PROXY=http://HOST_IP:53128
set HTTPS_PROXY=http://HOST_IP:53128
```

Browser Configuration

Firefox

1. Settings → General → Network Settings
2. Select "Manual proxy configuration"
3. HTTP Proxy: `HOST_IP`, Port: `53128`
4. SOCKS Host: `HOST_IP`, Port: `51080`, SOCKS v5

Chrome/Edge

Use system proxy settings or extensions like SwitchyOmega.

VPN Configuration

1. Obtain your VPN provider's `.ovpn` configuration file
2. Set environment variables:

   ```
   USE_VPN=true
   VPN_USERNAME=your_username
   VPN_PASSWORD=your_password
   VPN_OVPN_PATH=/path/to/config.ovpn
   ```

3. Start the service: `./start`

VPN Features

- **Auto-Reconnect**: Automatically reconnects on disconnect
- **Health Monitoring**: Periodic connectivity checks
- **Cache Invalidation**: Optional cache clear on VPN reconnect

Caching

How It Works

1. **Request Interception**: Proxy intercepts HTTP/HTTPS requests
2. **Cache Lookup**: Checks if response is already cached
3. **Freshness Check**: Validates cache freshness
4. **Response**: Serves from cache or fetches from origin
5. **Storage**: Caches valid responses for future use

Streaming Cache

Special handling for video/audio streaming:

- Chunk-based caching for partial content
- Range request support
- Configurable streaming domains
- Separate cache pool

Cache Invalidation

Automatic invalidation:

- Files older than `CACHE_MAX_AGE_DAYS`
- When cache exceeds `CACHE_MAX_SIZE_GB`
- On VPN reconnect (if configured)

Manual invalidation:

```
./cachectl invalidate
```

Security

Best Practices

1. **Network Restrictions**: Configure `ALLOWED_NETWORKS` to limit access
2. **Authentication**: Enable `PROXY_AUTH_ENABLED` for user authentication
3. **Firewall**: Use host firewall to restrict access

4. **VPN**: Enable VPN for privacy and geo-restriction bypass

Firewall Configuration

```bash
# Allow from specific network only
sudo ufw allow from 192.168.1.0/24 to any port 53128
sudo ufw allow from 192.168.1.0/24 to any port 51080
```

Troubleshooting

Service Won't Start

1. Check logs: `./logs/proxy.log`
2. Verify ports are not in use: `ss -tuln | grep -E '53128|51080'`
3. Check container runtime: `./init --check`

VPN Not Connecting

1. Verify VPN credentials in `.env`
2. Check `.ovpn` file path and permissions
3. Check VPN container logs: `$COMPOSE_CMD logs proxy-vpn`

Cache Not Working

1. Verify cache directory exists and is writable
2. Check Squid logs: `./logs/squid/cache.log`
3. Verify cache configuration: `./cachectl stats`

Connection Refused

1. Verify service is running: `./status`
2. Check firewall rules
3. Verify client is using correct IP and port

Development

Running Tests

```
./tests/run-tests.sh
```

Building Custom Images

```bash
podman build -t proxy-custom:latest .
```

Contributing

See CONTRIBUTING.md for contribution guidelines.

License

See LICENSE for license information.

Support

- **Issues**: GitHub Issues
- **Documentation**: See `docs/` directory for detailed documentation
  - Client Setup Guide — Configure any device
  - Android TV Guide — Android TV / Google TV setup
  - Browser Guide — Firefox, Chrome, Edge, Safari
  - Mobile Devices — iOS and Android
  - Network Modes — VPN vs no-VPN explained
  - VPN Setup — VPN configuration
  - Architecture — System design
  - Caching — Cache management
  - Troubleshooting — Common issues

Network Modes Explained

Mode 1: Containerized VPN (`USE_VPN=true`)

```
Client → Proxy → VPN Container → VPN Server → Internet
```

- VPN runs inside a container
- Complete isolation
- Best for dedicated proxy server

Mode 2: Host VPN Pass-through (`--host-vpn`)

```
Client → Proxy (host network) → Host VPN → VPN Server → Internet
```

- Uses host's existing VPN connection
- Containers share host network namespace
- Best when host already has VPN

Mode 3: No VPN (`--no-vpn` or `USE_VPN=false`)

```
Client → Proxy → Direct Internet
```

- No VPN routing

- Bridge network isolation
- Best for local caching only

Quick Comparison

Mode	Command	VPN Source	Network
Containerized	`./start` (with `USE_VPN=true`)	Container	VPN container
Host Pass-through	`./start --host-vpn`	Host system	Host network
No VPN	`./start --no-vpn`	None	Bridge

Tracked-Items + Status Documents

Per Helix Constitution §11.4.57. Each row links the tracked status / continuation document with its current Revision and Last-modified stamp (auto-derived from each document's §11.4.44 header). The canonical Issues / Fixed trackers are not present in this project and are omitted rather than fabricated.

Document	Last modified	Revision	Markdown	HTML	PDF
Continuation	2026-07-02T07:49:30Z	18	md	html	pdf
Feature Status	2026-07-01T15:30:00Z	3	md	html	pdf
Feature Status Summary	2026-07-01T15:30:00Z	3	md	html	pdf
Hardening Status	2026-07-01T13:26:00Z	3	md	html	pdf
Hardening Status Summary	2026-07-01T13:26:00Z	3	md	html	pdf
Security Status	2026-07-01T14:48:00Z	3	md	html	pdf
Security Status Summary	2026-07-01T14:48:00Z	3	md	html	pdf
Let's Encrypt Status	2026-07-01T11:42:00Z	3	md	html	pdf
Let's Encrypt Status Summary	2026-07-01T11:42:00Z	3	md	html	pdf

Document	Last modified	Revision	Markdown	HTML	PDF
VPN-LAN Integration Status	2026-07-01T19:05:00Z	3	md	html	pdf
VPN-LAN Integration Status Summary	2026-07-01T19:05:00Z	3	md	html	pdf
VPN-LAN Feature Status	2026-07-02T07:49:30Z	7	md	html	pdf
VPN-LAN Feature Status Summary	2026-07-02T07:49:30Z	7	md	html	pdf